



Utwórz plik Haskell o nazwie `cwiczenia_listy_krotki.hs`. Rozwiązania powinny zawierać deklaracje typów funkcji oraz przykładowe wywołania w komentarzach.

Dane startowe

```
temperatury :: [Double]
temperatury = [18.5, 21.0, 19.8, 23.4, 17.9, 22.2, 20.1]

produkty :: [(String, Double)]
produkty = [("monitor", 899.99), ("klawiatura", 149.90), ("mysz", 79.90), ("router", 249.00)]

punktyLab :: [(String, [Int])]
punktyLab = [("Nowak", [8, 9, 7]), ("Kowalska", [10, 10, 9]), ("Lis", [5, 6, 4])]

xsy :: [Int]
xsy = [2, 4, 6, 8]

ysy :: [Int]
ysy = [1, 3, 5, 7]
```

Zadanie 1. Operacje podstawowe na liście

Dla listy temperatury napisz wyrażenia lub funkcje, które zwrócą:

- liczbę pomiarów,
- trzy pierwsze pomiary,
- listę bez dwóch pierwszych pomiarów,
- najwyższą i najniższą temperaturę w postaci krotki.

Zadanie 2. Filtrowanie i mapowanie

Napisz funkcje:

- `ciepleDni :: [Double] -> [Double]` — pozostawia temperatury większe lub równe 20.0,
- `zaokraglone :: [Double] -> [Int]` — zamienia temperatury na liczby całkowite funkcją `round`,
- `rozniceOd20 :: [Double] -> [Double]` — oblicza różnicę każdego pomiaru względem wartości 20.0.

Zadanie 3. Listy z zakresów i list comprehensions

Utwórz funkcje:

- `wielokrotnosci3 :: Int -> [Int]` — zwraca wielokrotności liczby 3 od 3 do podanej granicy,
- `kwadratyNieparzystych :: Int -> [Int]` — zwraca kwadraty liczb nieparzystych z zakresu od 1 do `n`,
- `paryRozne :: [Int] -> [Int] -> [(Int, Int)]` — tworzy pary (x, y) , dla których $x \neq y$.

Zadanie 4. Krotki i listy krotek

Dla listy `produkty` przygotuj funkcje:

- `nazwyProduktow :: [(String, Double)] -> [String]` — zwraca tylko nazwy produktów,
- `drogieProdukty :: Double -> [(String, Double)] -> [(String, Double)]` — zwraca produkty droższe od podanej ceny,
- `cenaProduktu :: (String, Double) -> Double` — zwraca cenę pojedynczego produktu.

Zadanie 5. Dane studentów jako krotki z listą

Dla listy punktyLab napisz funkcje:

- `sumaPunktow :: (String, [Int]) -> Int` — zwraca sumę punktów jednego studenta,
- `wynikiKoncowe :: [(String, [Int])] -> [(String, Int)]` — zwraca nazwisko oraz sumę punktów każdego studenta,
- `zaliczyli :: [(String, [Int])] -> [(String, Int)]` — zwraca studentów, którzy uzyskali co najmniej 20 punktów.

Zadanie 6. zipWith i praca na dwóch listach

Dla list xsy i ysy napisz funkcje:

- `sumyPar :: [Int] -> [Int] -> [Int]` — sumuje elementy o tych samych indeksach,
- `iloczynyPar :: [Int] -> [Int] -> [Int]` — mnoży elementy o tych samych indeksach,
- `punkty2D :: [Int] -> [Int] -> [(Int, Int)]` — tworzy listę punktów na płaszczyźnie.

Zadanie dodatkowe

Napisz funkcję `raportProduktow :: [(String, Double)] -> [String]`, która dla każdego produktu tworzy napis w formacie: `nazwa ++ " kosztuje " ++ show cena`.